MREN 103 – Mechatronics and Robotics Design I
Mechatronics and Robotics Engineering Program
Queen's University, Kingston

Group #30
Laboratory #4 – Line Following with the MiniBot

Student Names: Brady Cabral and Emily Vargas-Penaranda
Instructor Name: Prof. Surgenor
Date Report Submitted: March 8, 2026

Summary:
A prewired MiniBot was used as an introduction to line-following sensors. Activities included: a) Identifying the left and right sensors, b) Finding the average left and right sensor values on white and black surfaces c) Producing a sensor plot for the means of calibration d) programming the MiniBot to follow a black line. The objectives of this lab include understanding the operating principles behind a line-following sensor, calibrating the latter, and programming the MiniBot to follow a black line with closed loop on/off control.

Hardware:
The setup for the MiniBot is shown in **Fig 1,** equipped with two infrared reflective line-following sensors mounted underneath the chassis. The MiniBot utilizes an Adafruit ItsyBitsy Microcontroller to read sensor signals and control the robot's motors and LEDs. The sensors are connected to the analog input pins of the Arduino, and output analog voltage is proportional to the amount of infrared light reflected from the surface beneath the robot. The emitter in each line following-sensor projects light toward the surface and the receiver detects the reflected light intensity. Testing the sensors by moving the robot forward and backwards over a black line yielded **Fig 2a.** and testing the sensors by moving them side to side over a black line yielded **Fig 2b**.

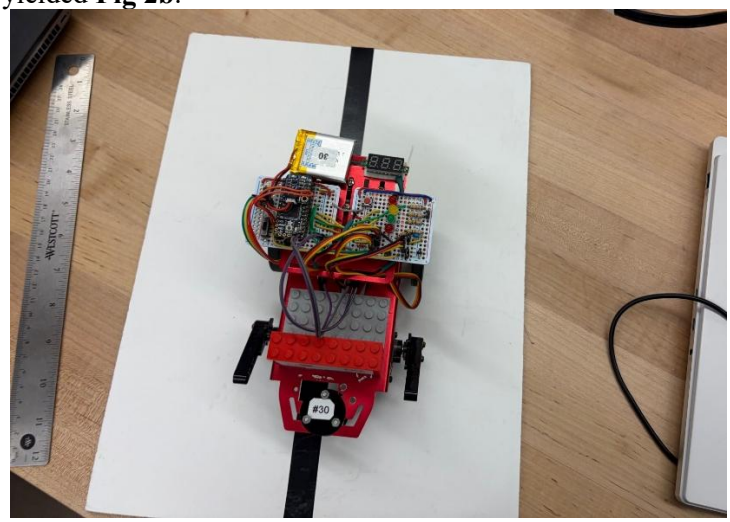

Fig 1. Setup of robot over black line testing sensors

Software:
The primary objective of the program **Group30Lab4LineFollowOnOff** shown in **Appendix A** is to follow a black line on the floor adjusting motor outputs to maintain a clear following. This works by first initializing all necessary variables such as the known pinouts for sensor and motors. Then the program waits for the button to be pressed and flashes the green LED whilst doing so. Afterwards the program tells the robot to continuously read the sensor data from the left and right and based on this determines what

moves to make. If one side detects the black line, it means it is leering in the wrong direction, and the bot turns the other direction to remain on the line. If both sensors detect black or none of them detect black the robot continues moving forward. This continues until the program is reset or the power runs out.

Results*:*

Table 1 showcases the voltage output from left and right sensors, placed in 3 different white surfaces and 3 different black surfaces.

Q1. The distance between internal sensors is approximately 1.9cm, and the thickness of the black line is also approximately 1.9cm. Since the sensor spacing is roughly equal to the line thickness, when the robot is placed at the center of the line, each sensor will be positioned on either side of the tape. This allows the robot to drift left or right depending on which sensor detects the black surface. Consequently, the robot is able to make adjustments to maintain alignment with the black tape.
[1] https://cdn.robotshop.com/media/g/gtd/rb-gtd-02/pdf/linefollowerdatasheet.pd

Q2. The collector current goes to 0 when sensing the distance is less than 0.7mm because at very small distances, the reflected light does not reach the phototransistor effectively due to the geometry of the emitter and receiver. In this fashion, as the sensing distance increases, the collector current decreases because the intensity of the reflected light also decreases with distance. The implication for this line following application is that the sensor must be placed at an appropriate distance above the surface. If the sensor is too far away or too close to the surface, the reflected signal becomes weak and unreliable, making it difficult for the sensor to actually distinguish between black and white surfaces.

Q3. According to the graph in [2] Figure 2, the normalized collector current peaks around 0.8mm (close to 1mm). When the sensor is too close (less than 0.7mm), or when it is too far (significantly above 1mm), the sensor's performance is poor. The measured distance between surface and sensor was approximately 2mm. Although this is more than the optimal distance, it is still within a reasonable operating range. Moreover, parallax and shakiness when taking the measurement could mean the distance between the surface and sensor is less than measured. The average of multiple measurements would likely reveal a more accurate measured distance.
[2] https://www.onsemi.com/pdf/datasheet/qre1113-d.pdf

Q4. The measurements in each column of **Table 1** are not significantly different from one another. The difference in each column can easily be accounted for natural variation in sensor readings (i.e. some noise in the sensors) and differences in initial sensor placements across white and black surfaces. Therefore, the calculated averages are reliable for the four conditions.

Q5. The outputs for left and right sensors are very similar, meaning that a single threshold value can be used for both sensors to differentiate between black and white surfaces. White surface values ranged from 240-293 mV, while black surface values ranged from 2648-2762 mV. A reasonable threshold value would lie between these two ranges, such as 1500 mV. If greater than this number, the detected surface is black. If less than this number, the detected surface is black.

All figures and tables need to be numbered and cited in the text.

Conclusions:

This lab demonstrated how infrared reflective sensors can be used for line detection and robotic navigation. Through calibration, it was observed that the sensors produce distinct voltage outputs for black and white surfaces, allowing a single threshold value to identify both black and white surfaces. Overall, this lab provided practical and theoretical knowledge on line following sensors and their application in closed-loop control in robotics.

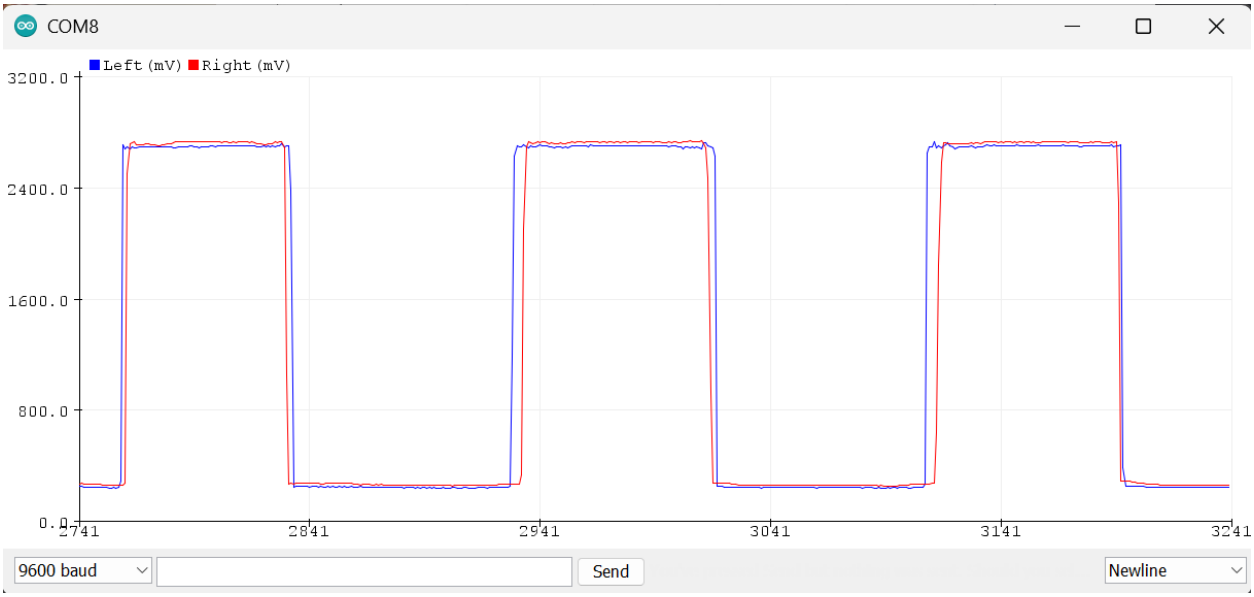

Fig 2a. Sensor plot from moving the robot back and forth across the black line

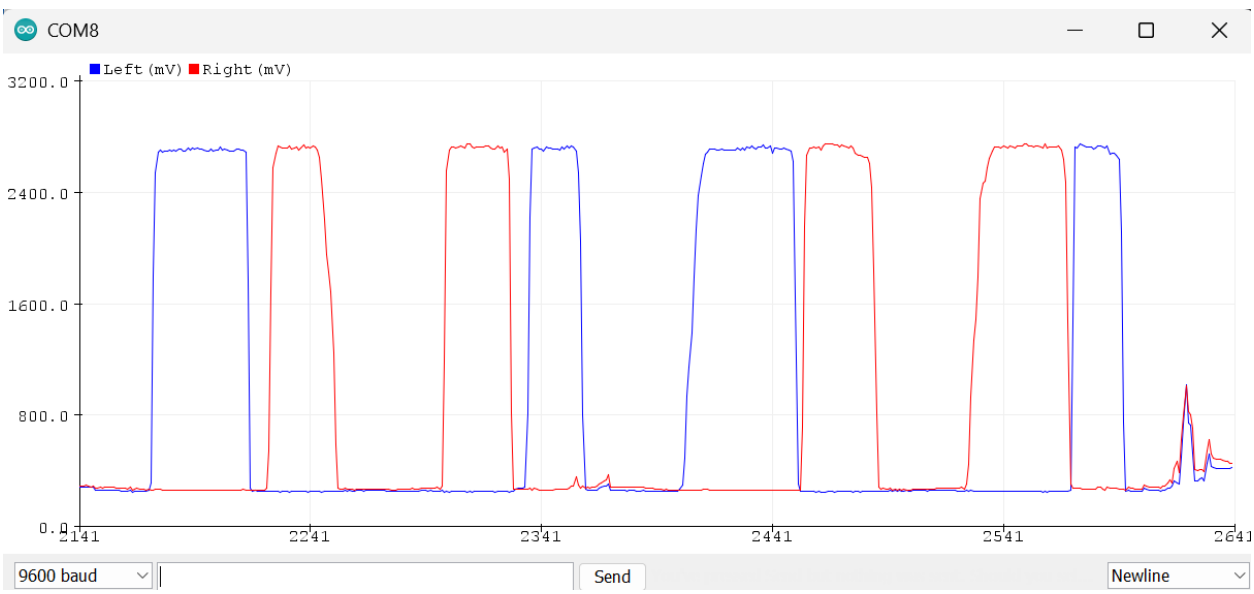

Fig 2b. Sensor plot from moving robot side to side across black line

Table 1. Recording measurements of sensor data from QRE113GR

Surface	White Surface		Black Tape	
Location	Left (mV)	Right (mV)	Left (mV)	Right (mV)
1	240	249	2648	2662
2	255	266	2762	2674
3	278	293	2668	2668
avg	258	269	2692	2668

Appendix A - Program Listing:

```
/*******************************************************
  Group30Lab4LineSensorCheck (v1.4)

  Brady Cabral, Emily Vargas-Penaranda (March 4
2026)

  Controls robot to keep it following black line
*******************************************************/



#define blcktp 2000
#define DELTA 10

#include <Servo.h>
Servo leftWheel;
Servo rightWheel;


// Pin Assignments
const int RED = 10;          //red LED Pin
const int GRN = 9;          //green LED Pin
const int YLW = 5;          //yellow LED Pin
const int BUTTON = 7;       //pushbutton Pin
int MOTOR_R = 3;        // right motor signal pin
int MOTOR_L = 4;        // left motor signal pin

const int LSENSOR = A2; // Left Sensor on Analog Pin
1
const int RSENSOR = A1; // Right Sensor on Analog
Pin 2
const boolean PLOT = false;  //true=plot sensor
reading; false=serial monitor output.
boolean loopbttn = true;

// Delta = speed above (+) or below (-) stop speed
(must be positive)
const int stopPulse = 149;  // stop speed for motors
(default = 155)
const int offset = 2;       // offset, slows right wheel
(default = 0)

//global variables
int lvalue = 0;  //left sensor value
int rvalue = 0;  //right sensor value



// Set-up Routine
void setup() {

// Initialize led pins as outputs.
  pinMode(GRN, OUTPUT);
  pinMode(YLW, OUTPUT);
  pinMode(RED, OUTPUT);

// Initialize button pins as inputs
  pinMode(BUTTON, INPUT);

// Initialize line following sensor pins as inputs
  pinMode(LSENSOR, INPUT);
  pinMode(RSENSOR, INPUT);

  //initialize motor control pins as servo pins
  leftWheel.attach(MOTOR_L);
  rightWheel.attach(MOTOR_R);

// Initialize serial and monitor
  Serial.begin(9600);
  runMotors(0,0);

}

// Main Routine
void loop() {

    while (loopbttn && digitalRead(BUTTON) == LOW) {
      digitalWrite(GRN, HIGH);
      delay(125);
      digitalWrite(GRN, LOW);
      delay(125);
    }
    loopbttn = false;

    //read the sensor value
    lvalue = analogRead(LSENSOR);
    rvalue = analogRead(RSENSOR);

    //map the values into millivolts (assuming 3000 mV
reference voltage)
    lvalue = map(lvalue,0,1023,0,3000);
    rvalue = map(rvalue,0,1023,0,3000);

    if (rvalue > blcktp && lvalue > blcktp) {
      turnOnLED(GRN);
      runMotors(DELTA, DELTA);
    }
    // Left side
    else if (rvalue > blcktp){
      turnOnLED(YLW);
      runMotors(DELTA, 0);

    }/* Right side */else if (lvalue > blcktp) {
      turnOnLED(RED);
      runMotors(0, DELTA);
    }/* Straddle */ else if (rvalue < blcktp && lvalue <
blcktp){

      turnOnLED(GRN);
```

```
      runMotors(DELTA,DELTA);
    }

    if(PLOT){                  // true means serial plotter
      Serial.print("Left(mV):");
      Serial.print(lvalue);
      Serial.print("\t");
      Serial.print("Right(mV):");
      Serial.println(rvalue);
      delay(40);
    }

}

//********** Functions (subroutines) ******************

// run robot wheels
void runMotors(int deltaL, int deltaR)
{
  int pulseL = (stopPulse + deltaL)*10;    //length of
pulse in microseconds
  int pulseR = (stopPulse + deltaR)*10;
  leftWheel.writeMicroseconds(pulseL);
  rightWheel.writeMicroseconds(pulseR);
}

// Turn on a single LED and turn others off
void turnOnLED(int COLOUR)
{
  digitalWrite(GRN, LOW);
  digitalWrite(YLW, LOW);
  digitalWrite(RED, LOW);
  digitalWrite(COLOUR, HIGH);
}
```